DuocUC | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

**TITULO**

# Informe de seguridad y calidad en el desarrollo de software

**ISY1102 - Seguridad y calidad en el desarrollo de software**

Sección 001V

Integrantes: Solange Argomedo Alfaro.

# INDICE

INDICE ..... 2
1. Introducción ..... 3
2. Criterios de calidad ..... 3
3. Plan de pruebas ..... 5
3.1 Estrategia de pruebas ..... 5
3.2 Tipos de prueba ..... 6
3.3 Criterios de aceptación ..... 7
3.4 Herramientas utilizadas ..... 8
4. Diseño de casos de prueba ..... 9
5. Ejecución de pruebas ..... 15
5.1 Resumen de ejecución ..... 17
5.2 Detalle de ejecución de casos ..... 18
5.3 Criticidad de incidentes ..... 20

## 1. Introducción

*El presente informe tiene como objetivo evaluar la seguridad y calidad del sistema web Atlas, una aplicación destinada a la administración de clientes, contratos, empresas y usuarios. El sistema permite gestionar información mediante distintas funcionalidades y niveles de acceso, por lo que resulta importante verificar su correcto funcionamiento y la protección de los datos que administra.*

*La elaboración de un plan de pruebas es necesaria para identificar posibles errores, vulnerabilidades y problemas de calidad que puedan afectar la operación del sistema. Mediante estas pruebas se busca comprobar que las funcionalidades respondan de manera adecuada, que los usuarios tengan acceso únicamente a las operaciones autorizadas y que la aplicación presente condiciones aceptables de usabilidad, rendimiento, estabilidad y confiabilidad.*

*Para desarrollar la evaluación se utiliza un entorno de pruebas local implementado con Docker, que integra el frontend, el backend y una base de datos PostgreSQL. Además, se emplean herramientas como SonarQube para analizar la calidad y seguridad del código fuente y GitHub para mantener el control de versiones del proyecto.*

*El informe se estructura en cinco apartados principales. Primero se establecen los criterios de calidad que debe cumplir el software. Luego se presenta el plan de pruebas, incluyendo la estrategia, los tipos de prueba, los criterios de aceptación y las herramientas utilizadas. Posteriormente se diseñan los casos de prueba y, finalmente, se documenta su ejecución, los resultados obtenidos y la criticidad de los incidentes identificados.*

## 2. Criterios de calidad

*Para evaluar el sistema web Atlas se establecen criterios de calidad orientados a comprobar su correcto funcionamiento, seguridad y confiabilidad. Estos criterios consideran las características de la aplicación y la importancia de proteger la información de clientes, contratos, empresas y usuarios.*

### *Calidad del software*

***Usabilidad:** El sistema debe presentar una interfaz clara e intuitiva, que permita a los usuarios navegar por sus funcionalidades, completar formularios y comprender los mensajes mostrados.*

***Rendimiento:*** *La aplicación debe responder en tiempos adecuados durante operaciones habituales, como iniciar sesión, consultar clientes y administrar contratos, evitando demoras que dificulten el trabajo.*

***Estabilidad:*** *El sistema debe mantener su funcionamiento durante el uso normal, sin cierres inesperados ni interrupciones frecuentes.*

***Confiabilidad:*** *Las operaciones deben producir resultados correctos y consistentes. Por ejemplo, al registrar o modificar información, los datos deben almacenarse y recuperarse correctamente.*

***Mantenibilidad:*** *El código fuente debe facilitar la identificación y corrección de defectos. Para evaluar este aspecto se utiliza SonarQube, considerando indicadores como errores detectados, duplicación de código y problemas de mantenibilidad.*

***Seguridad del software***

***Protección de datos:*** *Atlas debe proteger la información almacenada y evitar que usuarios no autorizados accedan a datos sensibles.*

***Control de acceso:*** *Las funcionalidades deben respetar los permisos asociados a cada rol. Un usuario no debe poder ejecutar acciones para las cuales no tenga autorización.*

***Manejo de errores:*** *El sistema debe informar los errores de manera comprensible, sin revelar contraseñas, claves de acceso, detalles internos de la base de datos u otra información sensible.*

***Integridad de la información:*** *Las operaciones de creación, modificación y eliminación deben respetar las validaciones correspondientes, evitando registros incorrectos o modificaciones no autorizadas.*

***Gestión de credenciales:*** *Las contraseñas y claves utilizadas por los servicios deben mantenerse protegidas y no exponerse innecesariamente en el código fuente.*

***Estándares técnicos y normativa***

***ISO/IEC 25010:*** *Se utiliza como referencia para evaluar características de calidad del software, tales como adecuación funcional, eficiencia de desempeño, usabilidad, fiabilidad, seguridad y mantenibilidad.*

***OWASP:*** *Se consideran sus recomendaciones de seguridad para aplicaciones web, especialmente en autenticación, control de acceso, validación de entradas y protección de información sensible.*

***Ley N.º 19.628 sobre Protección de la Vida Privada (Chile):*** *Se considera como referencia normativa para el tratamiento y resguardo de datos personales. Su aplicación concreta dependerá de la información que gestione el sistema y del marco legal vigente.*

***Buenas prácticas de desarrollo seguro:*** *Se promueve el control de versiones mediante GitHub, la revisión del código con SonarQube y la utilización de entornos de prueba separados de los entornos productivos.*

## 3. Plan de pruebas

### 3.1 Estrategia de pruebas

*Para evaluar el sistema Atlas utilizaremos una estrategia de pruebas mixta, es decir, combinaremos pruebas manuales con el uso de herramientas que nos ayudan a revisar el código.*

Las pruebas manuales y con Postman permiten revisar funciones como el inicio de sesión, la respuesta del servidor y la navegación. También se pueden planificar nuevas comprobaciones de clientes, contratos y permisos, que no se dan por aprobadas sin evidencia.

*También utilizaremos SonarQube, una herramienta que permite analizar el código y detectar posibles errores, problemas de seguridad y aspectos que se pueden mejorar. Esta herramienta nos ayudará a conocer el estado del código del frontend y backend.*

***Etapas del proceso:***

1. *Preparar el sistema Atlas utilizando Docker.*
2. *Revisar el código con SonarQube.*
3. *Diseñar los casos de prueba.*
4. *Ejecutar las pruebas en el sistema.*
5. *Guardar capturas de pantalla como evidencia.*
6. *Registrar los errores encontrados y proponer mejoras.*

***Priorización de las pruebas:***

*Primero revisaremos las funciones más importantes, como el inicio de sesión y los permisos de acceso, porque un error en estas áreas podría afectar la seguridad de la información. Después evaluaremos otras funciones, como el rendimiento y la facilidad de uso.*

***Criterios de entrada:***

*Para comenzar las pruebas, el sistema debe estar funcionando, la base de datos debe estar conectada y debemos contar con los usuarios necesarios para realizar las verificaciones.*

***Criterios de salida:***

*Las pruebas finalizarán cuando se hayan ejecutado los casos planificados, registrado sus resultados y guardado las evidencias correspondientes. Los errores encontrados se clasificarán según su importancia.*

## 3.2 Tipos de prueba

*Para evaluar la seguridad y calidad del sistema Atlas, realizaremos diferentes tipos de pruebas que nos permitirán comprobar su funcionamiento e identificar posibles problemas.*

***Pruebas funcionales:*** *Comprobaremos que las funciones principales del sistema trabajen correctamente, como iniciar sesión, consultar clientes, registrar contratos y utilizar las opciones disponibles según el tipo de usuario.*

***Pruebas no funcionales:*** *Revisaremos aspectos relacionados con la calidad del sistema, como su velocidad de respuesta, estabilidad y facilidad de uso.*

***Pruebas de seguridad:*** *Verificaremos que los usuarios solo puedan acceder a las funciones permitidas según su rol. También revisaremos los posibles problemas de seguridad identificados por SonarQube.*

Pruebas de integración: Permiten revisar que la página, el servidor y la base de datos trabajen juntos. En esta entrega se observó el uso del sistema, pero no se documentó una prueba completa de guardar y volver a consultar datos.

***Pruebas de usabilidad:*** *Revisaremos que las pantallas sean fáciles de entender, que los botones y formularios sean claros y que los mensajes ayuden al usuario a realizar sus tareas.*

***Pruebas de rendimiento:*** *Observaremos cuánto demora el sistema en responder al realizar acciones comunes, como iniciar sesión o consultar información.*

***Análisis estático del código:*** *Utilizaremos SonarQube para revisar el código del frontend y backend. Esta herramienta nos permitirá identificar errores, posibles problemas de seguridad y aspectos que se pueden mejorar sin necesidad de ejecutar cada función del sistema.*

### 3.3 Criterios de aceptación

*Para determinar si las pruebas realizadas al sistema Atlas son exitosas, estableceremos criterios de aceptación que nos permitan comparar los resultados esperados con los resultados obtenidos.*

***Funcionamiento:*** *Las funciones principales, como iniciar sesión, consultar clientes y gestionar contratos, deben realizarse correctamente y sin presentar errores.*

***Seguridad:*** *Los usuarios deben acceder solamente a las funciones permitidas según su rol. El sistema no debe permitir que un usuario sin autorización consulte o modifique información restringida.*

***Validación de datos:*** *Los formularios deben detectar datos incorrectos o campos obligatorios vacíos, mostrando mensajes que permitan al usuario corregirlos.*

***Rendimiento:*** *Para las operaciones comunes que mediremos, estableceremos como objetivo un tiempo de respuesta máximo de 3 segundos en el entorno local de pruebas.*

***Usabilidad:*** *Las pantallas deben ser comprensibles, los botones deben funcionar correctamente y los mensajes deben indicar claramente lo que ocurre.*

***Integridad de los datos:*** *La información registrada o modificada debe guardarse correctamente y mantenerse al volver a consultarla.*

***Clasificación de resultados:***

- ***Aprobado:*** *El sistema cumple con el resultado esperado.*
- ***Fallido:*** *El sistema presenta un error o no cumple con lo esperado.*
- ***Bloqueado:*** *No es posible ejecutar la prueba por un problema del entorno o una condición necesaria que no se cumple.*
- ***No ejecutado:*** *La prueba todavía no se ha realizado.*

*Se considerará que una prueba fue aprobada cuando cumpla con su resultado esperado y sus criterios de aceptación. Los casos fallidos deberán registrarse junto con su evidencia para analizar el problema encontrado.*

## 3.4 Herramientas utilizadas

*Para realizar las pruebas de seguridad y calidad del sistema Atlas, utilizamos diferentes herramientas que nos permiten ejecutar el proyecto, revisar su código y registrar los resultados.*

***Docker Desktop:*** *Lo utilizamos para ejecutar el sistema Atlas en nuestro computador. Nos permite iniciar el frontend, el backend y la base de datos PostgreSQL sin tener que instalar cada servicio por separado.*

***SonarQube:*** *Lo utilizamos por primera vez para analizar el código del frontend y backend. Esta herramienta nos ayuda a encontrar posibles errores, problemas de seguridad y aspectos del código que se pueden mejorar.*

Postman: Lo utilizamos para enviar solicitudes al backend de Atlas y comprobar sus respuestas. Probamos que el servidor estuviera disponible y que rechazara un inicio de sesión con datos incorrectos.

***Git y GitHub:*** *Los utilizamos para guardar el código del proyecto y mantener un registro de sus cambios. También nos permiten tener una copia del trabajo en un repositorio privado.*

***Git Bash:*** *Lo utilizamos para escribir comandos que nos permiten trabajar con Git, ejecutar instrucciones de Docker y realizar los análisis con SonarQube.*

***Visual Studio Code:*** *Lo utilizamos para revisar los archivos del proyecto y realizar cambios cuando sea necesario.*

***Navegador web:*** *Lo utilizaremos para ingresar a Atlas, comprobar sus funciones y tomar capturas de pantalla como evidencia de las pruebas.*

***PostgreSQL:*** *Es la base de datos que utiliza Atlas para almacenar la información del sistema. Nos permite comprobar que los datos registrados se guarden y puedan consultarse correctamente.*

***Recursos humanos:***

*Las pruebas serán realizadas por los integrantes del equipo de trabajo, quienes se encargarán de ejecutar los casos de prueba, registrar los resultados y preparar el informe.*

***Recursos técnicos:***

*Se utilizará un computador con Docker Desktop, Git Bash, Visual Studio Code, Postman y un navegador web. También se requiere acceso al código fuente de Atlas y a SonarQube.*

***Entorno de pruebas:***

*Las pruebas se realizarán en un entorno local, utilizando Docker para ejecutar los servicios del sistema. El frontend de Atlas se encuentra disponible en http://localhost:3351, el backend en http://localhost:4451 y SonarQube en http://localhost:9000.*

*Este entorno permite realizar las pruebas sin utilizar un servidor de producción.*

### 3.5 Organización y cronograma de pruebas

Preparación: iniciar Atlas y comprobar que frontend, backend y base de datos estén disponibles. Responsable: equipo de Atlas.

Revisión de código: consultar los resultados de SonarQube del frontend y backend. Responsable: equipo de Atlas.

Ejecución: realizar CP-01 a CP-07 y guardar las capturas. Responsable: equipo de Atlas.

Cierre: registrar resultados, incidentes y mejoras propuestas. Responsable: equipo de Atlas.

Fechas de cada actividad: completar según el registro real del equipo. No se indican fechas que no estén confirmadas.

## 4. Diseño de casos de prueba

### Relación entre requisitos y pruebas (trazabilidad)

Los códigos de requisitos incluidos en los casos son referencias usadas en este informe. Deben compararse con el ERS original antes de afirmar que se cubre el 100 % de los requisitos.

REQ-FUNC-01 → CP-01: inicio de sesión del administrador (aprobado).

REQ-SEG-01 → CP-02: inicio de sesión del editor (fallido).

REQ-NF-01 → CP-03: tiempo de respuesta de clientes (aprobado en una medición con caché).

REQ-CAL-01 → CP-04: análisis de código con SonarQube (realizado con observaciones).

REQ-NF-02 → CP-05: navegación del sistema (aprobado según registro de ejecución).

REQ-DISP-01 → CP-06: disponibilidad básica del backend con Postman (aprobado). Código de referencia de este informe, pendiente de cotejar con el ERS.

REQ-SEG-02 → CP-07: rechazo de credenciales incorrectas con Postman (aprobado). Código de referencia de este informe, pendiente de cotejar con el ERS.

Pendiente: cotejar estos requisitos con el ERS del proyecto y agregar casos si existen requisitos sin prueba. La cobertura completa del ERS aún no está verificada.

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-01** | **Caso de prueba CP-01: Inicio de sesión** | REQ-FUNC-01 |
| **DESCRIPCIÓN** | | |
| **Comprobar que un usuario registrado pueda ingresar correctamente al sistema Atlas utilizando su correo electrónico y contraseña.** | | |
| **PRECONDICIONES** | | |
| • **El sistema Atlas debe estar funcionando.**<br>• **El usuario debe estar registrado en el sistema.**<br>• **Debemos contar con un correo electrónico y una contraseña válidos para realizar la prueba.** | | |
| **PASOS** | | |
| 1. **Abrir el navegador web.**<br>2. **Ingresar a http://localhost:3351.**<br>3. **Ir a la pantalla de inicio de sesión.**<br>4. **Escribir el correo electrónico del usuario.**<br>5. **Ingresar su contraseña.**<br>6. **Presionar el botón para iniciar sesión.**<br>7. **Observar el resultado.** | | |
| **RESULTADO ESPERADO** | | |
| **El sistema debe permitir el ingreso del usuario y mostrar la pantalla principal de Atlas.** | | |
| **CRITERIO DE ACEPTACIÓN** | | |
| **La prueba se considera aprobada si el usuario puede iniciar sesión con sus datos correctos y acceder al sistema sin errores.** | | |
| **DATOS DE PRUEBA** | | |
| • **electrónico: usuario de prueba registrado en Atlas.**<br>• **Contraseña: contraseña válida del usuario de prueba.**<br>**Por seguridad, no se incluirán contraseñas reales en el informe**<br><br>**Tipo de prueba: Funcional: autenticación.** | | |

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| CP-02 | Inicio de sesión de usuario con rol Editor. | REQ-SEG-01 |

**DESCRIPCIÓN**

**Comprobar que un usuario creado con el rol Editor pueda iniciar sesión en el sistema Atlas utilizando su correo electrónico y contraseña.**

**PRECONDICIONES**

- **El sistema Atlas debe estar funcionando.**
- **Debe existir una cuenta Administrador.**
- **El Administrador debe haber creado un usuario con rol Editor.**

**PASOS**

1. **Iniciar sesión en Atlas con la cuenta Administrador.**
2. **Entrar a la sección Usuarios.**
3. **Crear un usuario nuevo con rol Editor.**
4. **Comprobar que aparezca en la lista de usuarios.**
5. **Cerrar la sesión del Administrador.**
6. **Intentar iniciar sesión con el correo y contraseña del Editor.**
7. **Observar el resultado.**

**RESULTADO ESPERADO**

**El usuario Editor debe poder iniciar sesión correctamente y acceder a las funciones que tiene permitidas según su rol.**

**CRITERIO DE ACEPTACIÓN**

**La prueba será aprobada si el usuario Editor puede ingresar a Atlas con sus credenciales correctas, sin recibir mensajes de error.**

**DATOS DE PRUEBA**

- **Nombre de usuario: editor_prueba**
- **Correo electrónico: editor.atlas@duocuc.cl**
- **Rol: Editor**
- **Contraseña: contraseña creada únicamente para la prueba.**

**Tipo de prueba: Funcional y seguridad: acceso de usuario.**

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-03** | Tiempo de carga de clientes | REQ-NF-01 |

**DESCRIPCIÓN**

**Comprobar cuánto demora el sistema Atlas en cargar la pantalla de clientes, para verificar que responda en un tiempo adecuado.**

**PRECONDICIONES**

- **El sistema Atlas debe estar funcionando.**
- **El usuario debe haber iniciado sesión.**
- **La conexión con la base de datos debe estar disponible**

**PASOS**

1. **Iniciar sesión en Atlas con la cuenta Administrador.**
2. **Abrir las herramientas del navegador presionando F12.**
3. **Seleccionar la pestaña Network (Red).**
4. **Hacer clic en la sección Clientes de Atlas.**
5. **Revisar el tiempo que demora la solicitud para cargar los clientes.**
6. **Registrar el tiempo obtenido y guardar una captura de pantalla.**

**RESULTADO ESPERADO**

**La consulta de clientes debe completarse correctamente, sin errores y en un tiempo adecuado**

**CRITERIO DE ACEPTACIÓN**

**Para esta evaluación se establece como objetivo que la solicitud de clientes demore como máximo 3 segundos en el entorno local de pruebas**

**DATOS DE PRUEBA**

- **Usuario: Administrador.**
- **Módulo: Clientes.**
- **Entorno: Atlas ejecutado localmente con Docker.**
- **Tiempo máximo establecido: 3 segundos**

**Tipo de prueba: No funcional: rendimiento.**

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-04** | Análisis de código con SonarQube | REQ-CAL-01 |

**DESCRIPCIÓN**

**Revisar la seguridad y calidad del código del sistema Atlas utilizando SonarQube, para identificar posibles errores, problemas de seguridad y aspectos que se pueden mejorar.**

**PRECONDICIONES**

7. **El sistema Atlas debe estar funcionando.**
8. **Tener Docker funcionando.**
9. **Tener SonarQube instalado y funcionando.**
10. **Contar con el código fuente del frontend y backend de Atlas.**
11. **Tener ambos proyectos configurados para su análisis en SonarQube.**
12. **El usuario debe haber iniciado sesión.**
13. **La conexión con la base de datos debe estar disponible**

**PASOS**

1. **Abrir SonarQube en http://localhost:9000.**
2. **Ingresar con la cuenta de SonarQube.**
3. **Seleccionar el proyecto del frontend de Atlas.**
4. **Revisar los errores, problemas de seguridad y aspectos de calidad detectados.**
5. **Repetir la revisión con el proyecto del backend.**
6. **Registrar los resultados y guardar capturas de pantalla.**

**RESULTADO ESPERADO**

**SonarQube debe completar el análisis de ambos proyectos y mostrar los posibles problemas encontrados en el código**

**CRITERIO DE ACEPTACIÓN**

**El análisis debe finalizar correctamente y permitir revisar los resultados del frontend y backend. Los problemas detectados deben quedar registrados para su posterior revisión.**

**DATOS DE PRUEBA**

- **Herramienta: SonarQube.**
- **Proyecto 1: Frontend de Atlas.**
- **Proyecto 2: Backend de Atlas.**
- **Entorno: SonarQube ejecutado localmente mediante Docker.**
- **Código analizado: código fuente de Atlas.**

**Tipo de prueba: Análisis estático: calidad y seguridad.**

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-05** | Navegación por las opciones de Atlas | REQ-NF-02 |
| **DESCRIPCIÓN** | | |
| **Comprobar si el usuario puede encontrar y utilizar fácilmente las principales opciones del sistema Atlas, como Clientes, Contratos y Gestión de Usuarios.** | | |
| **PRECONDICIONES** | | |
| • **El sistema Atlas debe estar funcionando.**<br>• **El usuario debe haber iniciado sesión como Administrador.**<br>• **La pantalla principal debe estar disponible.** | | |
| **PASOS** | | |
| 1. **Iniciar sesión en Atlas con la cuenta Administrador.**<br>2. **Observar las opciones disponibles en el menú principal.**<br>3. **Hacer clic en la sección Clientes.**<br>4. **Volver al menú y seleccionar Contratos.**<br>5. **Ingresar a Gestión de Usuarios.**<br>6. **Comprobar si los nombres de las opciones son claros y si es fácil cambiar entre las secciones.**<br>7. **Registrar el resultado y guardar una captura de pantalla.** | | |
| **RESULTADO ESPERADO** | | |
| **El usuario debe poder identificar las principales opciones del menú y navegar entre las secciones sin dificultad.** | | |
| **CRITERIO DE ACEPTACIÓN** | | |
| **Las opciones deben tener nombres claros, ser visibles y permitir al usuario acceder a las secciones correspondientes sin confusión.** | | |
| **DATOS DE PRUEBA** | | |
| • **Usuario: Administrador.**<br>• **Secciones: Clientes, Contratos y Gestión de Usuarios.**<br>• **Entorno: Atlas ejecutado localmente con Docker.**<br>• **Herramienta: Navegador web.**<br>**Tipo de prueba: No funcional: usabilidad.** | | |

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-06** | Disponibilidad del backend con Postman | REQ-DISP-01 |
| **DESCRIPCIÓN** | | |
| **Comprobar que el servidor de Atlas se encuentre disponible y responda a una solicitud desde Postman.** | | |
| **PRECONDICIONES** | | |
| **Atlas debe estar funcionando con el backend iniciado. Postman debe estar instalado.** | | |
| **PASOS** | | |
| **Abrir Postman.**<br>**Crear una solicitud GET.**<br>**Escribir http://localhost:4451.**<br>**Presionar Send.**<br>**Revisar el estado y la respuesta del servidor.** | | |
| **RESULTADO ESPERADO** | | |
| **El servidor debe responder con HTTP 200 y un mensaje que confirme su funcionamiento.** | | |
| **CRITERIO DE ACEPTACIÓN** | | |
| **Se aprueba si devuelve HTTP 200 y una respuesta que indique que Atlas está funcionando.** | | |
| **DATOS DE PRUEBA** | | |
| **Herramienta: Postman.**<br>**Método: GET.**<br>**Dirección: http://localhost:4451.**<br>**Respuesta obtenida: HTTP 200 en 56 ms; {"name":"Atlas","status":"ok"}.**<br>**Tipo de prueba: disponibilidad básica del backend.** | | |

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-07** | Rechazo de credenciales incorrectas con Postman | REQ-SEG-02 |
| **DESCRIPCIÓN** | | |
| **Comprobar que Atlas no permita iniciar sesión con un usuario inexistente y una contraseña incorrecta.** | | |
| **PRECONDICIONES** | | |
| **Atlas y su backend deben estar funcionando. Postman debe estar disponible. No se utilizarán contraseñas reales.** | | |
| **PASOS** | | |
| **Abrir Postman.**<br>**Seleccionar POST y escribir http://localhost:4451/autenticacion/login.**<br>**En Body, elegir raw y JSON.**<br>**Ingresar los datos ficticios de prueba.**<br>**Presionar Send y revisar el resultado.** | | |
| **RESULTADO ESPERADO** | | |
| **El sistema debe rechazar el acceso con HTTP 401 y el mensaje «Credenciales inválidas».** | | |
| **CRITERIO DE ACEPTACIÓN** | | |
| **Se aprueba si el servidor rechaza las credenciales incorrectas y no permite iniciar sesión.** | | |
| **DATOS DE PRUEBA** | | |
| **Herramienta: Postman.**<br>**Método: POST.**<br>**Dirección: http://localhost:4451/autenticacion/login.**<br>**Datos: nombre_usuario = usuario_prueba_inexistente; password = clave_incorrecta_123.**<br>**Respuesta obtenida: HTTP 401 en 71 ms; {"error":"Credenciales inválidas"}.**<br>**Tipo de prueba: seguridad y validación de inicio de sesión.** | | |

# 5. Ejecución de pruebas

## 5.1 Resumen de ejecución

Se documentaron siete casos de prueba para evaluar el inicio de sesión, el rendimiento, el análisis estático, la usabilidad, la disponibilidad del backend y el rechazo de credenciales incorrectas.

- Casos diseñados: 7. Casos ejecutados: 7.
- Resultados: 5 aprobados (CP-01, CP-03, CP-05, CP-06 y CP-07), 1 fallido (CP-02) y 1 aprobado con observaciones (CP-04). En total, 6 pruebas aprobaron, una de ellas con observaciones. No hubo casos bloqueados ni sin ejecutar.
- El caso CP-02 requiere una revisión adicional para confirmar por qué el usuario Editor no pudo ingresar. SonarQube encontró aspectos del código que también necesitan revisión.

## 5.2 Detalle de ejecución de casos

A continuación, se registran los resultados observados en cada caso. Las evidencias corresponden a las capturas obtenidas durante las pruebas; se deben adjuntar al documento si aún no están insertadas. Las pruebas CP-06 y CP-07 se realizaron con Postman y sus capturas deben incorporarse a la carpeta de evidencias.

| CASO | DESCRIPCIÓN | RESULTADO | EVIDENCIA |
|---|---|---|---|
| **PRUEBAS FUNCIONALES, SEGURIDAD Y NO FUNCIONALES** | | | |
| **CP-01** | Inicio de sesión con usuario Administrador. | APROBADO: El usuario ingresó correctamente a Atlas y visualizó la pantalla principal. | *EVIDENCIA* |
| **CP-02** | Inicio de sesión con usuario Editor. | FALLIDO: El usuario Editor fue creado, pero el inicio de sesión mostró «Credenciales inválidas» incluso después de cambiar la contraseña. Causa pendiente de confirmar. | *EVIDENCIA* |
| **CP-03** | Tiempo de respuesta de la consulta de clientes. | APROBADO: Con sesión iniciada, la solicitud «clientes» registró HTTP 304 en 30 ms (0,03 s), dentro del límite de 3 s. La respuesta utilizó caché. | *EVIDENCIA* |
| **CP-04** | Análisis estático del frontend y backend con SonarQube. | APROBADO CON OBSERVACIONES: Ambos Quality Gates aprobaron. Frontend: 1 bug, 35 code smells, 11,2 % duplicación. Backend: 3 security hotspots pendientes, 6 code smells, 26,3 % duplicación. No se reportaron vulnerabilidades. | *EVIDENCIA* |
| **CP-05** | Navegación y usabilidad de Atlas. | APROBADO: Se pudo navegar por Inicio, Clientes, Contratos y Usuarios; los nombres y opciones de las pantallas fueron claros. | *EVIDENCIA* |
| **CP-06** | Disponibilidad del backend con Postman. | APROBADO: Solicitud GET a http://localhost:4451. HTTP 200 en 56 ms; respuesta de Atlas con estado «ok». | *EVIDENCIA* |
| **CP-07** | Rechazo de credenciales incorrectas con Postman. | APROBADO: Solicitud POST a /autenticacion/login. HTTP 401 en 71 ms y mensaje «Credenciales inválidas». | *EVIDENCIA* |

Fecha de referencia de las evidencias: 24/10/2026. Confirmar las fechas de cada captura antes de entregar.

Responsable de las pruebas: Solange Argomedo Alfaro (confirmar si participaron otros integrantes).

Nota: CP-03 se midió con respuesta HTTP 304, que reutiliza caché. Una medición sin caché permitiría evaluar la carga inicial.

Evidencias: adjuntar las capturas correspondientes a CP-01 a CP-07 o pegar el enlace de la carpeta «Evidencias EP3 Atlas» de Google Drive, con acceso de lectura para el docente.

## 5.3 Criticidad de incidentes

La siguiente escala permite clasificar los incidentes según su efecto en el sistema. La gravedad de los hallazgos que aún no se han investigado es provisional.

| CASO | DESCRIPCIÓN | IMPACTO | EVIDENCIA |
|---|---|---|---|
| **CRITICO** | El defecto impide completamente el funcionamiento del sistema o de una funcionalidad esencial para el negocio. Bloquea el flujo principal. | Muy alto, el sistema no puede ser usado | Requiere remediación inmediata |
| **GRAVE** | El defecto afecta funcionalidades importantes, pero el sistema aún funciona parcialmente. Existe un workaround pero la experiencia se ve comprometida. | Alto, afecta reas principales del usuario | Debe corregirse con urgencia |
| **MODERADO** | El defecto afecta funcionalidades secundarias o casos específicos sin detener el flujo principal | Medio, el usuario puede trabajar, pero con molestias o limitaciones | Se corrige en próxima iteración |
| **LEVE** | Defecto de apariencia, estética o texto el cual no afecta la funcionalidad. | Bajo, no impacta la operación | Se corrige según disponibilidad |

*Para cada incidente identificado se determina el nivel de impacto que tiene en el sistema*

| CASO | DESCRIPCIÓN | RESULTADO | IMPACTO |
|---|---|---|---|
| **PRUEBAS FUNCIONALES Y DE SEGURIDAD** | | | |
| **CP-02** | Inicio de sesión con usuario Editor. | El usuario no pudo iniciar sesión. Se mostró «Credenciales inválidas» después de crear la cuenta y cambiar la contraseña; falta confirmar la causa. | GRAVE |
| **ANÁLISIS ESTÁTICO DE CÓDIGO** | | | |
| **CP-04** | Hallazgos de SonarQube en frontend y backend. | Se detectó 1 bug, 3 security hotspots pendientes de revisión y problemas de mantenibilidad y duplicación. Los Quality Gates aprobaron. | MODERADO |

CP-06 y CP-07 no generaron incidentes: ambas respuestas coincidieron con lo esperado. En CP-07, el código 401 es correcto porque se utilizaron credenciales incorrectas.

Observación: el error HTTP 401 observado inicialmente en Clientes correspondía a una sesión cerrada; al repetir la prueba con sesión iniciada no se reprodujo. Por eso CP-03 no se clasifica como incidente.

Acciones recomendadas: revisar los registros del backend y la creación de credenciales del usuario Editor; revisar el bug y los tres security hotspots en SonarQube; reducir la duplicación de código.

Los security hotspots son puntos para revisar, no vulnerabilidades confirmadas. La criticidad es provisional hasta comprobar el impacto de cada hallazgo.

## 5.4 Análisis ético y normativo de los resultados

Atlas maneja información de clientes y usuarios. Por eso es importante que cada persona acceda solo a los datos que necesita y que las contraseñas se mantengan protegidas. Estas medidas ayudan a cuidar la privacidad y la confianza de quienes usan el sistema.

La Ley N.º 19.628 sobre protección de datos personales se considera como referencia para tratar la información con cuidado. También se toman como guía las buenas prácticas de OWASP y los criterios de calidad de ISO/IEC 25010. Mencionar estas referencias no significa que se haya realizado una auditoría legal o una certificación.

En las pruebas, el usuario Editor no logró iniciar sesión. Esto afecta el acceso esperado a su cuenta y debe investigarse. SonarQube mostró tres puntos de seguridad pendientes de revisión en el backend; no corresponde llamarlos vulnerabilidades confirmadas sin analizarlos. Se recomienda corregir el problema de acceso, revisar los hallazgos y repetir las comprobaciones después de los cambios.

## Conclusión y mejoras pendientes

Las siete pruebas permitieron conocer el estado de Atlas. Se verificó el acceso del administrador, se observó una dificultad con la cuenta Editor, se registró una respuesta de 30 ms con caché en la consulta de clientes y se revisó el código de frontend y backend con SonarQube. Ambos proyectos aprobaron el control de calidad configurado, aunque tienen observaciones que requieren atención. Además, Postman confirmó que el backend respondió con HTTP 200 en 56 ms y que rechazó credenciales incorrectas con HTTP 401 en 71 ms.

Como próximos pasos se propone revisar el acceso del Editor, repetir la medición de clientes sin caché, estudiar el bug del frontend y los tres puntos de seguridad del backend, y comparar los casos con el ERS para confirmar la cobertura de requisitos. No se afirma que estas mejoras ya estén implementadas.